

Analítica de Big Data | FACEN - Universidad Nacional de Asunción | 2026

Subnivel: 1.1 Qué es Big Data y para qué sirve

Guía de Funciones: 1.1 Big Data — R y Python

En este subnivel aún no hay código de análisis de datos como tal, pero sí exploraremos cómo verificar el entorno y hacer una primera lectura de datos grandes.

R — Verificación del entorno y lectura inicial

```
# Verificar version de R
R.version$version.string

# Instalar y cargar paquetes esenciales de Big Data en R
if (!require("data.table")) install.packages("data.table")
if (!require("arrow"))      install.packages("arrow")
library(data.table)
library(arrow)

# Leer un CSV grande con data.table (mucho más rápido que read.csv)
# data.table usa todos los cores disponibles automáticamente
dt <- fread("dataset_grande.csv")

# Ver estructura
str(dt)          # Tipos de variables y primeras filas
dim(dt)          # Filas y columnas
head(dt, 10)     # Primeras 10 filas
summary(dt)      # Estadísticas descriptivas básicas
```

```
# Verificar memoria usada
object.size(dt) # Tamaño en bytes
format(object.size(dt), units = "MB") # En megabytes

# Número de núcleos disponibles (para paralelismo)
parallel::detectCores()
```

Explicación línea a línea

1. `R.version$version.string`: Confirma que R está actualizado.
2. `fread()`: Lee CSV grandes usando múltiples núcleos. Es 10-100x más rápido que `read.csv()`.
3. `str(dt)`: Muestra la estructura interna: tipos de columnas, primeras observaciones.
4. `dim(dt)`: Retorna `[n_filas, n_columnas]`. Fundamental para conocer la escala.
5. `object.size()`: Cuánta RAM ocupa el dataset. Si supera el 80% de tu RAM, necesitás procesamiento por chunks.

Python — Verificación del entorno y lectura inicial

```
import sys
import polars as pl
import pandas as pd

# Verificar versiones
print(f"Python: {sys.version}")
print(f"Polars: {pl.__version__}")
print(f"Pandas: {pd.__version__}")

# Leer CSV con Polars (más rápido que Pandas para datos grandes)
df = pl.read_csv("dataset_grande.csv")

# Exploración inicial
print(df.shape) # (n_filas, n_columnas)
print(df.dtypes) # Tipos de datos por columna
print(df.head(10)) # Primeras 10 filas
```

```
print(df.describe())      # Estadísticas descriptivas

# Ver uso de memoria aproximado
print(f"Memoria estimada: {df.estimated_size('mb'):.2f} MB")

# Lazy evaluation: solo ejecuta cuando se llama .collect()
# Útil para datasets que no caben en memoria
lazy_df = pl.scan_csv("dataset_grande.csv")
resultado = lazy_df.filter(pl.col("columna") > 100).collect()
```

Explicación línea a línea

1. `pl.read_csv()`: Polars lee CSVs usando Rust internamente, ~3-5x más rápido que pandas.
2. `df.shape`: Tupla (filas, columnas). Primer diagnóstico del dataset.
3. `df.dtypes`: Lista de tipos. Detecta si columnas numéricas fueron leídas como texto.
4. `df.estimated_size("mb")`: Tamaño en RAM. Si supera 4 GB en una máquina de 8 GB, usar lazy evaluation.
5. `pl.scan_csv()`: Lazy mode — construye el plan de ejecución sin leer datos todavía.
6. `.collect()`: Ejecuta el plan optimizado. Polars optimiza automáticamente el orden de operaciones.